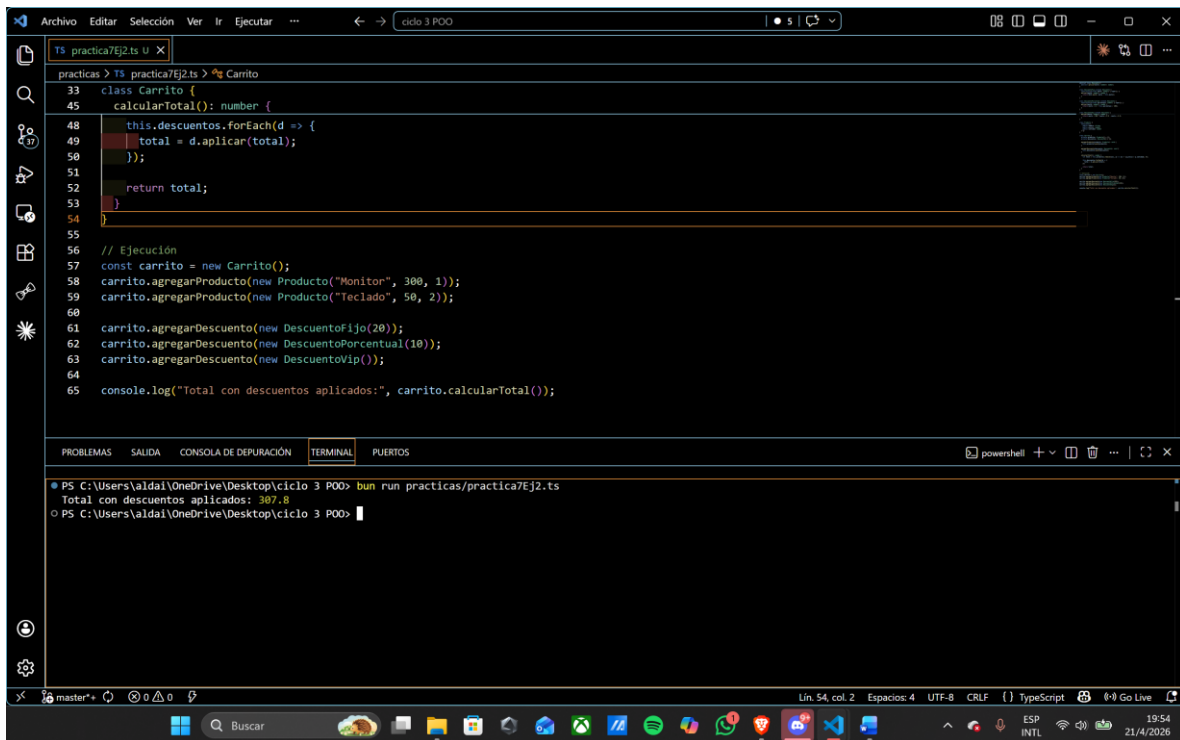




```
practicas > TS practica7Ej2.ts > Carrito
33 class Carrito {
45   calcularTotal(): number {
48     this.descuentos.forEach(d => {
49       total = d.aplicar(total);
50     });
51   }
52   return total;
53 }
54
55
56 // Ejecución
57 const carrito = new Carrito();
58 carrito.agregarProducto(new Producto("Monitor", 300, 1));
59 carrito.agregarProducto(new Producto("Teclado", 50, 2));
60
61 carrito.agregarDescuento(new DescuentoFijo(20));
62 carrito.agregarDescuento(new DescuentoPorcentual(10));
63 carrito.agregarDescuento(new DescuentoVip());
64
65 console.log("Total con descuentos aplicados:", carrito.calcularTotal());
```

```
PS C:\Users\aldal\OneDrive\Desktop\ciclo 3 POO> bun run practicas/practica7Ej2.ts
Total con descuentos aplicados: 307.8
PS C:\Users\aldal\OneDrive\Desktop\ciclo 3 POO>
```

Polimorfismo: Se ve reflejado cuando el Carrito recorre su lista de descuentos. Aunque todos están guardados como el tipo genérico Descuento, al llamar a aplicar(), el programa ejecuta la lógica específica de la clase que corresponda (Fijo, Porcentual o VIP) sin que el carrito tenga que saber cuál es cuál.

Ausencia de condicionales: No usamos if o switch para preguntar el tipo de descuento porque cada objeto es responsable de su propia lógica. Al invocar el método, el lenguaje ya sabe qué comportamiento ejecutar. Esto evita que el código se llene de ramas lógicas difíciles de mantener.

Agregar nuevos descuentos: Es tan simple como crear una nueva clase que extienda de Descuento. No necesitas tocar ni una sola línea del código del Carrito ni de los otros descuentos. El sistema está "abierto" para crecer pero "cerrado" a modificaciones en su núcleo.

Interacción entre objetos: El Carrito actúa como un contenedor que suma los precios de los Productos. Luego, le pasa ese resultado a los objetos Descuento de forma encadenada. Cada objeto recibe un valor, lo transforma y lo devuelve, permitiendo que múltiples beneficios se apliquen uno tras otro de manera ordenada.